



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica

Corso di Laurea Magistrale in Informatica

TESI DI LAUREA

Studio Cross-Sectional sulla Prevalence dei Code Smells in Progetti Quantum

RELATORE

Prof. **Fabio Palomba**
Dott. **Stefano Lambiase**
Università degli Studi di Salerno

CANDIDATO

Francesco Paolo D'Antuono
Matricola: 0522501767

Anno Accademico 2024-2025

Questa tesi è stata realizzata nel



Go Beyond...PLUS ULTRA!!!

Abstract

Lo sviluppo software è un'attività complessa che dipende sia da aspetti tecnici che da sociali. In questa tesi mi concentrerò sugli aspetti tecnici, poiché, in alcuni casi, per rispettare le scadenze prefissate di un progetto software, si tende ad introdurre il cosiddetto *Technica Debt*. Ciò si rivela spesso una scelta sbagliata, in quanto i problemi che non sono stati risolti oggi, un domani si riproporanno in maniera amplificata. Inoltre, lo studio si concentrerà su quelli che sono progetti *Quantum*, cioè progetti nei quali si utilizza il cosiddetto *Quantum Computing*.

In merito a ciò, la letteratura ha definito i Quantum-specific Code Smells, che vengono definiti come *scelte di progettazione e implementazione di basso livello da parte degli sviluppatori durante lo sviluppo e la manutenzione del quantum software*. Lo studio di questa tesi si concentrerà sull'individuazione dei Code Smells in progetti *Quantum*, effettuando anche un'analisi su slice temporali dei medesimi progetti utilizzati. Inoltre, andrò ad analizzare possibili dipendenze che ci possono essere tra coppie di Code Smells.

Per raggiungere l'obiettivo, è stato costruito un dataset di progetti *Quantum* e eseguito un tool su ognuno di essi, denominato Q-Spire, per ricavare i *Code Smells* presenti in ogni progetto e anche nelle slice temporali. Inoltre, con i risultati ottenuti, è stato utilizzato il modello statistico *cross-sectional* che ha permesso di capire quali fossero gli smells più comuni all'interno dei progetti *Quantum* e confrontare i medesimi per capire se ci sono dipendenze tra i *Code Smells* rilevati, attraverso il calcolo della *Prevalence* e della *Prevalence Odds Ratio*.

I risultati mostrano che la maggior parte dei progetti presenta lo smell *ROC (Repeated set of Operations on Circuit)* con il 75% dei progetti. Inoltre ci sono alcune dipendenze, tra cui spicca quella tra *IQ (Initialization of Qubits differently from $|0\rangle$)* e *IM (Intermediate Measurements)* con un valore molto alto di 24,89.

Indice

Elenco delle figure	2
Elenco delle tabelle	3
1 Introduzione	4
1.1 Contesto Applicativo	4
1.2 Motivazioni e Obiettivi	5
1.3 Risultati Ottenuti	5
1.4 Struttura della Tesi	6
2 Background e Stato dell'Arte	7
2.1 Quantum Computing	7
2.1.1 Quantum Software Engineering	7
2.1.2 Strumenti per il Quantum	8
2.2 Technical Debt	9
2.2.1 Code Smells	9
2.2.2 Quantum-specific Code Smells	10
2.3 Q-Spire	12
2.4 Modello Statistico	13
2.4.1 Modello Cross-Sectional	13
3 Metodologia	15
3.1 Obiettivo del Lavoro e Metodo Generale	15
3.2 Costruzione del Dataset	17
3.3 Divisione in Slice Temporali	19
3.4 Utilizzo del Modello Statistico	20
4 Risultati	22
4.1 Costruzione del Dataset ed Esecuzione del Tool	22
4.2 Divisione in Slice Temporali	24
4.3 Utilizzo del Modello Statistico	25
4.3.1 Prevalence	25
4.3.2 POR	26
5 Conclusioni	31
5.1 Riassunto dei Contenuti	31
5.2 Lavori Futuri	31

Elenco delle figure

4.1	<i>Esempio risultati Q-Spire</i>	22
4.2	<i>Esempio risultati script counter.py</i>	23
4.3	<i>results.xlsx</i>	23
4.4	<i>dataset_quantum_code_smells.xlsx</i>	23
4.5	<i>dataset_quantum_code_smells_sliced.xlsx</i>	24
4.6	<i>Prevalence dei Quantum Code Smells</i>	25
4.7	<i>Trends della Prevalence dei Quantum Code Smells</i>	26
4.8	<i>POR dei Quantum Code Smells</i>	27
4.9	<i>Trends della POR dei Quantum Code Smells</i>	29
4.10	<i>Trends della POR dei Quantum Code Smells</i>	30

Elenco delle tabelle

2.1	Quantum Code Smells	10
2.2	Categorie di modelli cross-sectional	13
4.1	Esempio righe <i>dataset_quantum-enabled_projects.xlsx</i>	22

Capitolo 1

Introduzione

1.1 Contesto Applicativo

Negli ultimi anni, il quantum computing è in rapida crescita, alimentato dall'interesse economico, politico e sociale che porta questa nuova branca dell'informatica, basata sui concetti di superposizione ed entanglement della fisica quantistica.

Durante il MIND Innovation Week [1], è stato discusso il futuro del computing, stimando che entro il 2030, il quantum computing avrà un valore di mercato che raggiungerà i 500 miliardi di euro.

Tuttavia, tutte le problematiche riscontrate nel software classico si possono riproporre anche in questo nuovo contesto, poiché la qualità del codice e la manutenzione di esso dipendono spesso dalla bravura e dalla conoscenza delle persone che lavorano ai progetti software.

Uno dei concetti che analizzeremo in questa tesi sono i Code Smells, ovvero segnali sintomatici di possibili problemi strutturali nel codice. Ovviamente, ciò che è stato riscontrato nel software classico non può essere direttamente applicato al Quantum, poiché ci sono concetti di base differenti che possono essere adattati ma non ricopiati. Ecco perché sono stati individuati i *Quantum-specific Code Smells*, cioè i Code Smells individuati all'interno di progetti Quantum.

1.2 Motivazioni e Obiettivi

L'obiettivo principale di questa tesi di ricerca è di analizzare i Code Smells all'interno di progetti *Quantum*, cioè progetti che sfruttano i concetti di base della fisica quantistica, per evolvere la potenza computazionale, superando di gran lunga quella dei supercomputer.

Ho pensato di realizzare questa tesi di ricerca perché sempre più progetti si stanno affacciando nel mondo del Quantum Computing e credo che oltre ad aiutarmi nell'acquisire conoscenze in questo ambito, permetterà di aprire nuove frontiere su queste tematiche sempre più diffuse nella comunità di sviluppo software. Dunque, lo scopo della tesi è proprio quello di capire quali problemi possono sorgere all'interno del Quantum Computing e le motivazioni dietro a queste problematiche.

Oltre a individuare gli smell all'interno dei progetti *Quantum*, lo studio si incentra anche sull'analisi dell'evoluzione dei progetti Quantum, dividendo i progetti in slice temporali e l'individuazione di dipendenze tra coppie di *Code Smells* utilizzando un modello statistico, denominato *cross-sectional*.

Inoltre, studi di questo tipo specifico non sono presenti ancora in letteratura e dunque la mia tesi potrebbe essere un buon punto di partenza per poi sviluppare altri lavori di questo tipo.

1.3 Risultati Ottenuti

I risultati ottenuti dalla tesi sono molto soddisfacenti. Infatti, sono riuscito a calcolare la *Prevalence* dei Code Smells in progetti Quantum, individuando come smell più presente *ROC (Repeated set of Operations on Circuit)* con il 75% di progetti. Andando a vedere anche i trends della *Prevalence*, nelle slice temporali si nota un aumento della presenza degli smell all'interno dei progetti.

Poi, sono riuscito a determinare la *POR* per ogni coppia di smell. Tra le varie associazioni individuate, quella più significativa risulta essere la coppia *IQ-IM*, dove il valore è di 24,89. Questa associazione può essere spiegata da una dipendenza tecnica reale, dovuta al fatto che un programmatore quantistico eseguendo misurazioni intermedie (IM), altera lo stato dei qubit e porta a reinizializzare i qubit in stati diversi rispetto allo stato standard $|0\rangle$ (IQ).

Infine, ho svolto uno studio temporale, in un range di circa 2 anni, prendendo di riferimento come data iniziale il 2020-01-01. I risultati hanno mostrato che tendenzialmente il "calore" della POR cresce. Ciò significa che più passa il tempo, più gli smell si presentano e in qualche modo dipendono tra di loro, rispettando però i valori delle dipendenze riscontrati sulle analisi dell'ultima slice temporale.

Tutto il lavoro svolto è stato reso disponibile in una repo GitHub¹ che spiega come è stato svolto il lavoro, con la possibilità di replicarlo.

¹https://github.com/CpDant/Studio_Cross-Sectional_sulla_Prevalence_dei_Code_Smells_in_Progetti_Quantum_Replication_Package

1.4 Struttura della Tesi

In questa sezione vengono mostrati tutti i capitoli, in maniera sintetica, di cui si compone il presente lavoro di tesi. In ognuno di questi vengono estesi i concetti già citati nelle sezioni precedenti di questo capitolo. Ecco di seguito la lista dei capitoli:

- **Capitolo 2: Background e stato dell'arte**, illustra le opere e le scoperte della letteratura fino ad oggi, sugli aspetti tecnici del Software e su come ci sia bisogno sempre di più di riconoscere le problematiche all'interno di questo nuovo tipo di software, per studiare delle possibili soluzioni.
- **Capitolo 3: Metodologia**, mostra il metodo generale e gli step utilizzati per raggiungere l'obiettivo principale.
- **Capitolo 4: Risultati**, illustra i risultati degli step singoli e dunque del lavoro di tesi.
- **Capitolo 5: Conclusioni**, fornisce una sintesi della ricerca e introduce ad alcuni possibili lavori futuri.

Capitolo 2

Background e Stato dell'Arte

2.1 Quantum Computing

Il Quantum Computing è un nuovo paradigma di calcolo che utilizza i principi definiti dalla fisica quantistica, per svolgere una computazione in maniera totalmente diversa rispetto ai computer classici [25]. Infatti, il Quantum Computing utilizza i qubits, l'unità base di informazione quantistica che può esistere in una sovrapposizione di 0 e 1 (denominata *superposition*).

Per manipolare l'informazione quantistica, bisogna utilizzare le porte quantistiche (quantum gates), che possono applicare operazioni sia su un singolo qbit che su più qubits simultaneamente.

Il Quantum Computing è fatto da circuiti quantistici. Dunque, un programma quantistico è composto da porte unitarie, applicate in un ordine specifico, per descrivere operazioni unitarie.

L'esecuzione di un circuito quantistico è svolta da dispositivi quantistici reali o simulatori, che permettono la creazione di stati base e l'applicazione delle operazioni, secondo l'ordine definito dal circuito.

2.1.1 Quantum Software Engineering

Il *Quantum Software Engineering (QSE)* è l'applicazione dei concetti di Software Engineering al Quantum Software, dunque, anche in questo caso lo sviluppo software deve seguire un processo specifico che viene descritto da varie fasi, adottando un approccio ingegneristico nello sviluppo di software quantistico.

Gli studi sono molto recenti su questa nuova branca dell'ingegneria del software. Il primo vero paper che ha esposto questo concetto in maniera dettagliata è il *Talavera Manifesto* di Piattini et al. [34], dove, oltre ad esporre i principi fondamentali del QSE, essi hanno esposto i principali campi di ricerca che dovevano essere analizzati, per colmare il gap di conoscenza che esiste tra quantum computer scientists e ingegneri del software.

Colui che ha fornito una definizione formale del QSE è Jianjun Zhao, che nel 14 luglio 2020 ha pubblicato un overview sul Quantum Software Engineering [42], definendolo in questo modo: "*Quantum software engineering is the use of sound engineering principles for the development, operation, and maintenance of quantum software and the associated document to obtain economically quantum software that is reliable and works efficiently on quantum computers.*". Ovviamente, in questo paper vengono riportati tutti i concetti che già conosciamo sull'ingegneria del software (ciclo di vita del software, etc.).

Un altro paper importante è quello di Moguel et al. [23], nel quale si è voluto esporre la necessità di imparare dalla storia, comparando il periodo storico attuale con quello degli anni '60, che ha visto protagonista i computer classici. In questo modo si è giustificato l'importanza dell'esistenza del QSE.

In campo italiano, De Stefano et al. [11] hanno condotto uno studio sistematico per capire a che punto si è con le ricerche sul QSE. Dallo studio, si è notato che le ricerche sul QSE si concentrano maggiormente sul software testing. Tuttavia c'è del potenziale per distribuire in maniera più equa le ricerche sulle diverse dimensioni del QSE.

Lo sviluppo di programmi quantistici presenta varie sfide. El Aoun et al. [16] hanno condotto uno studio empirico per capire queste sfide dal punto di vista dello sviluppatore. Ciò che è stato fatto è il mining di piattaforme come GitHub per identificare le domande più frequenti sul QSE. Lo studio ha mostrato che spesso le domande che vengono poste sono perlopiù sulla teoria di base dei programmi quantistici, mostrando la necessità di approfondire gli studi teorici su questi argomenti.

2.1.2 Strumenti per il Quantum

Ci sono vari strumenti che sono stati realizzati per riuscire a costruire un programma quantistico.

Uno dei più importanti è *Qiskit* [35], un framework open-source sviluppato da IBM, che permette di definire, simulare ed eseguire circuiti quantistici, utilizzando un linguaggio simile a Python. Le alternative sviluppate da Google e Microsoft sono rispettivamente *Cirq* [14], che possiede un maggiore controllo sul posizionamento dei qubit e sulla topologia del circuito, e *Q#* [22], che adotta un approccio più formale, poiché utilizza un linguaggio indipendente.

Essendo che ci sono ancora limitazioni di hardware, si utilizzano i simulatori per avere un ambiente in grado di supportare questi programmi. Strumenti come *Qiskit Aer* e *Cirq Simulator* consentono di testare circuiti quantistici reali.

Si hanno a disposizione anche ambienti di sviluppo integrati e facilmente accessibili come *IBM Quantum Platform* [15], strumento ideale per scrivere codice Qiskit.

2.2 Technical Debt

Il *technical debt* (TD) si riferisce a artefatti non completi o task posticipati che costituiscono un "debito", che porterà a costi aggiuntivi durante l'evoluzione e la manutenzione del software. Il concetto trae origine da una metafora finanziaria: così come un debito economico comporta il pagamento di interessi nel tempo, un technical debt genera un costo aggiuntivo nel ciclo di vita del software, non solo di tipo economico.

Il primo a definire questo concetto è stato Cunningham [10], che espose un'esperienza specifica, nella quale ha sviluppato un sistema di gestione del portafoglio. In questa esperienza, ci furono dei problemi di implementazione che hanno portato ad avere un debito da riparare.

Da quel momento in poi, sono stati condotti numerosi studi sul technical debt, ognuno con uno scopo differente. Avgeriou et al. [5] hanno scoperto che, sebbene il TD portasse inizialmente vantaggi, col passare del tempo può aumentare il costo dei cambiamenti. Martini et al. [20] hanno dimostrato che l'esistenza dei TD nei sistemi software è inevitabile.

Esistono vari tool per identificare i TD. Avgeriou et al. [6] hanno condotto uno studio di confronto di ben 9 tool, evidenziando le caratteristiche principali di ognuno di essi. Il migliore nel contesto Quantum si è rivelato essere SonarQube.

2.2.1 Code Smells

Una particolare forma di technical debt sono i *Code Smells*, scelte di progettazione e implementazione di basso livello da parte degli sviluppatori durante lo sviluppo e la manutenzione del software.

Negli ultimi anni, i ricercatori si sono soffermati molto su questo argomento, poiché risulta essere una delle cause principali del technical debt all'interno di prodotti software, esaminando le cause, evoluzioni e impatto sul software (Chatzigeorgiou and Manakos [8]; Arcoverde et al. [4]; Tufano et al. [41, 40]; Palomba et al. [27]).

Inoltre, sono stati realizzati degli studi per sviluppare metodi per la detection di Code Smells, sia di tipo euristico, utilizzando metriche del codice strutturali oppure semplici contenuti testuali e version history (Moha et al. [24], Tsantalis e Chatzigeorgiou [39], Palomba et al. [29, 28]) sia basati sul machine-learning (Arcelli Fontana et al. [3], Di Nucci et al. [12], Pecorelli et al. [31, 30, 32, 33]). Entrambi hanno i propri pro e contro, e la ricerca in questo ambito è ancora una sfida aperta. Alcuni ricercatori stanno esplorando approcci alternativi, come il deep learning (Liu et al. [18], Lin et al. [17]).

2.2.2 Quantum-specific Code Smells

Gli studi sui *Quantum-specific Code Smells* sono ancora agli inizi. Infatti, solo nel 2022, Openja et al. [26] hanno iniziato ad analizzare i technical debt in progetti di tipo Quantum, notando che c'è una forte correlazione tra gli errori all'interno del codice e errori di progettazione o di mancanza di conoscenza da parte di chi lavora a quel software.

I primi a trattare i Code Smells in Quantum software sono stati Chen et al. [9], che con la loro ricerca sono stati capaci di definire in maniera rigorosa un insieme di 8 Quantum Code Smells, utilizzando un tool di detection sviluppato da loro, denominato *QSMELL*. Di seguito elencheremo in una tabella i cosiddetti *The smelly eight*:

Tabella 2.1: Quantum Code Smells

Best Practice	Nome Smell	Acronimo	Descrizione
<i>Running a circuit in hardware</i>			
“Getting a circuit to run on hardware”	Use of Customized Gates	CG	Qualsiasi gate personalizzato è scomponibile in operatori nativi del framework. Questa scomposizione richiede un numero significativamente maggiore di operatori rispetto a una soluzione equivalente basata esclusivamente su operatori integrati.
“Using Circuit-Operation to reduce circuit size”	Repeated set of Operations on Circuit	ROC	A causa di vincoli tecnologici e fisici, il numero di operazioni che può essere eseguito su un computer quantistico è limitato. Pertanto l'intero algoritmo dovrebbe essere preparato in modo tale da minimizzare il numero di insiemi ripetuti di operazioni da eseguire.
<i>Inefficient circuits</i>			
“Use sweeps when possible”	Non-parameterized Circuit	NC	I dispositivi reali operano in modo condiviso. Per ridurre il traffico di comunicazione e evitare code per differenti valori iniziali, il circuito dovrebbe essere progettato parametrico in modo da permettere di fornire insieme i diversi valori iniziali.
<i>Erroneous circuits</i>			

Best Practice	Nome Smell	Acronimo	Descrizione
“Short gate depth”	Long Circuit	LC	I gate quantistici e le misurazioni sono soggetti a errori (in particolare per il rumore quantistico). Maggiore è la profondità del circuito e/o più ampio è, maggiore è la probabilità di alterare il comportamento previsto del circuito quantistico.
“Terminal Measurements”	Intermediate Measurements	IM	Le misurazioni influenzano lo stato dell'intero sistema, rendendolo più vulnerabile a errori. Perciò, le misurazioni devono essere posticipate come ultima operazione nel circuito per evitare la propagazione di errori.
“Keep qubits busy”	Idle Qubits	IdQ	Con la tecnologia attuale è possibile garantire la correttezza di uno stato solo per brevi periodi. Lasciare i qubit inattivi per troppo tempo favorisce la perdita di informazioni quantistiche, mettendo a rischio i risultati del circuito.
“Delay initialization of qubits”	Initialization of Qubits differently from $ 0\rangle$	IQ	Mantenere la coerenza di uno stato quantistico eccitato è tecnologicamente difficile. Pertanto, inizialmente è meglio mantenerlo nello stato fondamentale (cioè $ 0\rangle$) il più a lungo possibile.
“Qubit picking”	No-alignment between the Logical and Physical Qubits	LPQ	La topologia dei qubit reali influenza il comportamento del circuito. I risultati possono variare in base alla configurazione fisica dei qubit. Pertanto, non allineare i qubit logici con quelli fisici adeguati può portare a risultati meno accurati.

Di recente è stato pubblicato un altro studio condotto da Malhotra et al. [19], dove è stato esposto un altro tool per la detection di Quantum Code Smells, denominato *QNet*, che utilizza il deep learning. Nel dettaglio, il modello sviluppato è riuscito a superare i risultati raggiunti fino ad ora con un accuracy del 92,86

2.3 Q-Spire

Per l'analisi dei progetti open-source è stato utilizzato un nuovo tool ideato dal prossimo dott. Alfieri, denominato *Q-Spire*. Esso è un nuovo strumento open-source progettato per l'analisi e l'identificazione dei *Code Smells*, all'interno di progetti Quantum. La differenza sostanziale che si ha rispetto ai tool precedenti è che permette di rivelare gli smells per un progetto intero, rispetto agli altri che lavorano per singolo script. Il tool permette di fare due tipi di analisi:

- **Analisi dinamica:** Basata su una struttura di classi modulare e gerarchica, che garantisce la scalabilità e facilita l'integrazione di nuove logiche di rilevamento. Il design architettonico è basato sul modello Factory, incentrato su una classe astratta centrale Detector. I singoli rilevatori sono implementati come sottoclassi distinte che estendono Detector;
- **Analisi statica:** Basata su una simulazione del comportamento del codice, permette di fornire informazioni dettagliate su come si comporterebbe il codice senza un programma completo ed eseguibile. Ciò consente al tool di identificare e analizzare con precisione potenziali problemi nascosti all'interno di funzioni o classi, fornendo un'analisi completa.

Oltre alle sue capacità di rilevamento primarie, lo strumento incorpora una metodologia avanzata basata su un modello linguistico di grandi dimensioni (LLM). Questa integrazione consente agli utenti di richiedere spiegazioni dettagliate e approfondimenti su specifici difetti rilevati, offrendo una comprensione più approfondita dei problemi sottostanti e fornendo indicazioni su potenziali soluzioni e best practice. Di seguito riporto il link della repository GitHub di Q-Spire¹.

¹<https://github.com/Riccardoalfieri2003/Q-Spire>

2.4 Modello Statistico

Un altro strumento importante che è stato utilizzato per la tesi di ricerca è il modello statistico. Esso risulta essere un modello matematico che incorpora un insieme di ipotesi statistiche relative alla generazione di dati campione. Dunque, rappresenta il processo di generazione dei dati. Viene introdotto in questo contesto poiché risulta essere fondamentale per l'analisi e la comprensione dei dati che andremo a derivare sui *Community Smells* nei capitoli successivi.

In letteratura, esiste tanto materiale che tratta i modelli statistici. Per esempio, Gallucci et al. hanno scritto un libro [13], in cui parlano dei vari tipi di modelli statistici che possono essere utilizzati per l'analisi dei dati, nel campo sociale-psicologico. In questo libro vengono citati anche dei software che consentono di facilitare il lavoro statistico come R o SAS. Invece, Carpita e Maurizio hanno scritto un libro [7] dove è stata misurata la qualità del lavoro all'interno di cooperative sociali.

Inoltre, nell'ambito dell'Ingegneria del Software, è stata sviluppata una tesi di ricerca da Masiero [21], in cui è stato utilizzato un modello statistico, con dei dati derivati dai vari progetti software. L'analisi di questi dati ha permesso di valutare (tramite anche delle metriche di valutazione) la qualità del software.

2.4.1 Modello Cross-Sectional

Nella nostra tesi di ricerca utilizzeremo il modello statistico *cross-sectional*. Esso è un tipo di modello in cui si raccolgono dati da molti individui diversi in un punto nel tempo [38, 37]. Inoltre, risulta essere un modello semplice ed economico per la raccolta dei dati e l'identificazione di correlazioni.

I modelli cross-sectional possono essere categorizzati a seconda della natura dell'insieme dei dati e del tipo di dati ricercati [37]. La Tabella 2.2 riporta tutte le categorie di modelli cross-sectional.

Tabella 2.2: Categorie di modelli cross-sectional

#	Categoria	Scopo
1	Descriptive	Descrive le caratteristiche di una popolazione.
2	Analytical	Investiga le associazioni tra le variabili.
3	Community Survey/Population-Based Survey	Raccoglie informazioni su una popolazione o su un sottoinsieme di essa.
4	Prevalence Study	Determina la proporzione di una popolazione con una specifica caratteristica.
5	Occupational or Environmental	Esamina gli effetti di determinate esposizioni professionali o ambientali.
6	Occupational or environmental	Genera ipotesi per ricerche future.

Uno degli elementi chiave del modello *cross-sectional* risulta essere il *Relative Risk (RR)*, cioè la probabilità che un soggetto, appartenente a un gruppo esposto a determinati fattori, sviluppi la malattia, rispetto alla probabilità che un soggetto, appartenente a un gruppo non esposto, sviluppi la stessa malattia. Si potrebbe riassumere ciò che è stato definito in precedenza con la seguente formula matematica:

$$RR = I(esposti)/I(nonesposti)$$

dove I rappresenta l'incidenza, cioè il rapporto tra i nuovi casi di malattia durante un periodo di tempo e le persone a rischio all'inizio del periodo. Il RR può assumere tre significati differenti:

- **$RR = 1$** , il fattore di rischio è ininfluente sulla comparsa della malattia;
- **$RR > 1$** , il fattore di rischio è implicato nel manifestarsi della malattia;
- **$RR < 1$** , il fattore di rischio difende dalla malattia (fattore di difesa).

Un altro elemento chiave di questo modello è il *Prevalence Odds Ratio (POR)*, anche chiamato *Odds Ratio (OR)*. Esso è un dato statistico che misura il grado di correlazione tra due fattori. A differenza del RR che è uno studio prospettico, l'*Odds Ratio* viene utilizzato negli studi caso-controllo, cioè quelli che identificano fattori che possono contribuire a una condizione, quindi non è necessaria la raccolta dei dati nel tempo. Per calcolare gli OR si utilizza la seguente formula:

$$OR = \frac{odds_E}{odds_{-E}}$$

dove il numeratore indica l'odds² della malattia tra soggetti esposti e il denominatore indica l'odds della malattia tra soggetti non esposti. Anche l' OR assume tre significati differenti:

- **$OR = 1$** , l'odds di esposizione nei sani è uguale all'odds di esposizione nei malati, cioè il fattore in esame è ininfluente sulla comparsa della malattia;
- **$OR > 1$** , il fattore in esame può essere implicato nella comparsa della malattia (fattore di rischio);
- **$OR < 1$** , il fattore in esame è una difesa contro la malattia (fattore protettivo).

Sono stati condotti alcuni studi molto importanti, nei quali è stato utilizzato il modello *cross-sectional*, uno di questi è stato sviluppato da Al Omari et al. [2], in cui si è analizzata la relazione tra il COVID-19 e il DAS³, con l'utilizzo di un sondaggio che ha permesso di acquisire le informazioni di 6 nazioni differenti. Lo studio ha fatto emergere che i continui lockdown dovuti dal COVID-19, hanno impattato in maniera negativa sui giovani, aumentando il livello di DAS.

Nell'ambito che invece è più pertinente alla nostra tesi di ricerca, Shrivastava et al. hanno prodotto un articolo scientifico nel 2012 [36], nel quale è stato analizzato se ci fosse una correlazione tra le figure professionali nello sviluppo di un software e i loro problemi di salute. I risultati di questo studio hanno fatto notare che più ore passiamo a lavorare con il computer, più il rischio di problemi di vista o di postura aumenta. Inoltre, anche la postazione che si utilizza per lavorare influisce tanto sui problemi di postura.

²Rapporto tra la probabilità p di un evento e la probabilità che tale evento non accada

³Acronimo che indica le tre maggiori malattie mentali che possono affliggere una persona (depressione, ansia e stress)

Capitolo 3

Metodologia

3.1 Obiettivo del Lavoro e Metodo Generale

L'obiettivo di questo lavoro è stato capire e individuare come i progetti Quantum vengono sviluppati e pattern possibili di smells all'interno dello sviluppo, andando ad indagare nel dettaglio le possibili motivazioni che ci possono essere dietro a queste scelte di progettazione del software. Per fare ciò, ho valutato attraverso *Q-Spire* (spiegato nella sezione 2.3) gli smells all'interno dei progetti, andando poi ad utilizzare il modello cross-sectional per individuare statistiche che potessero descrivere i dati raccolti. In particolare si utilizzeranno i concetti di *Prevalence* e *Prevalence Odds Ratio* (anche chiamata semplicemente *Odds Ratio*), per capire quali fossero gli smells più presenti nei progetti e le possibili associazioni tra di essi. Inoltre, si è voluto analizzare l'evoluzione temporale dei progetti per derivare possibili pattern comuni negli smells e determinare la presenza di essi nel tempo. Per capire bene quali fossero gli obiettivi, ho formulato le seguenti domande di ricerca:

Domande di ricerca

RQ1: Qual è la *Prevalence* dei Code Smells nei progetti Quantum?

RQ2: Come varia nel tempo la *Prevalence* dei Code Smells in progetti Quantum?

Queste prime due domande di ricerca riguardano il calcolo della *Prevalence* di Code Smells all'interno dei progetti Quantum. Tutto ciò è stato fatto per capire quali fossero i Code Smells più presenti all'interno dei progetti analizzati ed avere un'analisi più dettagliata di come essi si evolvessero nel tempo, evidenziando possibili trend e pattern comuni.

Domande di ricerca

RQ3: Quali sono le relazioni tra differenti Code Smells in progetti Quantum-enabled

RQ4: Come variano nel tempo le relazioni tra differenti Code Smells in progetti Quantum?

Le altre due domande di ricerca si concentrano sullo studio delle dipendenze dei Code Smells a coppie, utilizzando il modello statistico Cross-Sectional e quindi calcolando la *Prevalence Odds Ratio* (*POR*) sui risultati ottenuti dal tool.

Per raggiungere gli obiettivi e rispondere alle domande di ricerca, il lavoro è stato diviso in 3 step fondamentali:

1. **Costruzione del Dataset:** In questa fase, si è cercato di raccogliere e filtrare i progetti open-source, per caratteristiche specifiche, in modo tale da poter essere eseguiti da Q-Spire.
2. **Divisione in Slice Temporali:** In questa fase andremo a dividere i commit dei progetti in slice temporali specifiche, per capire l'evoluzione dei progetti nel corso del tempo. Dunque, avremo più versioni dei progetti per capirne l'effettiva evoluzione e individuare possibili pattern di comportamento dei *Code Smells*.
3. **Utilizzo del Modello Statistico:** Verrà utilizzato il modello *cross-sectional*, il quale permette di calcolare la Prevalence e la *Prevalence Odds Ratio*. È stato utilizzato uno script Python, che, dato in input una coppia di smells, calcola quanto sono associate e questo procedimento è stato effettuato per ogni coppia di smells (senza ripetizione). I risultati sono stati salvati in un dataset e poi sono state prodotte due matrici diagonali che mostrano graficamente i risultati ottenuti.

3.2 Costruzione del Dataset

Per prima cosa si è costruito il dataset dei progetti open-source. Per farlo, ho scelto dei criteri che potessero essere sfruttati per le analisi fatte in seguito, che sono:

- **Disponibilità del progetto:** il progetto dovrebbe essere pubblico e accessibile tramite le API di GitHub;
- **Caratteristiche:** il progetto deve essere in Qiskit e in linguaggio Python;
- **Numero di commit:** il progetto deve avere un certo numero di commit (almeno 30), distribuiti in un range temporale di almeno 6 mesi.
- **Numero di stelle:** il progetto deve avere un certo numero di stelle, in questo modo siamo sicuri che si tratta di progetti open-source mantenuti e consistenti, poiché, essendo il quantum computing alle sue fasi primordiali, buona parte dei progetti risultano essere dei progetti giocattolo oppure supporto per la formazione che, nel nostro caso, sono superflui e inutilizzabili.

Dopo un'attenta analisi dei progetti open-source, da un insieme di 294 progetti, sono passato a 91 progetti, che possono essere sfruttati per l'analisi, presenti nel file *dataset_quantum-enabled_projects.xlsx* (i progetti all'interno del dataset hanno un flag che permette di capire quali sono stati presi in considerazione).

Dopo aver raccolto il set di progetti open-source, ho collaborato con il prossimo dott. Alfieri, il quale ha lavorato allo sviluppo di *Q-Spire* per la detection di Quantum-specific Code Smells per la sua tesi, facendo da tester per il suo progetto. Dunque, dopo essermi accertato che il tool funzionasse in maniera corretta, mi è bastato utilizzare lo script all'interno del progetto denominato StaticMapped-FolderDetection.py, passando come path all'interno del codice quello della cartella principale della repository da analizzare. Il tool scansiona in automatico tutti i file all'interno del progetto per ricercare gli script Python nei quali è possibile analizzare i circuiti quantistici, sia in maniera statica che dinamica.

Ovviamente, prima di avviare lo script, mi sono dovuto creare un virtual environment tramite il seguente comando *"python -m venv venv"*, che mi ha permesso di utilizzare la versione più recente di Python (3.13), installando le librerie di cui avevo bisogno per analizzare le repository. In alcuni casi, ci sono stati problemi di versioning delle librerie che non hanno permesso la corretta esecuzione del tool, ciò è dovuto al fatto che in alcune repository, anche se sono aggiornate a versioni recenti, utilizzano ancora versioni deprecated di classi e/o metodi che vanno in conflitto con la versione più recente del tool.

Per la raccolta dei dati ho ideato uno script Python, denominato *counter.py*, che mi ha permesso di prelevare le informazioni in maniera automatizzata dai risultati forniti dal tool.

Lo script deve essere utilizzato in questo modo *python counter.py {pathRisultatiDetection}* e permette di contare tutte le occorrenze degli smell che vengono rilevati dal tool e mostrarle come output sul terminale.

Infine, tutti i dati sono stati salvati all'interno di un dataset, denominato *dataset_quantum_code_smells.xlsx*.

3.3 Divisione in Slice Temporali

Il secondo step del lavoro di tesi mi ha consentito di analizzare l'evoluzione nel corso del tempo di questo nuovo tipo di software. Lo scopo principale è capire se essi si comportano come i software classici oppure hanno comportamenti anomali o differenti che potrebbero portare a nuove scoperte interessanti su come si evolve il Quantum software, andando a riconoscere possibili pattern nei *Code Smells*.

Per riuscire a raggiungere questo obiettivo, ho definito delle regole per la divisione dei progetti in slice temporali, che sono le seguenti:

- **Data di inizio:** 2020-01-01;
- **Data fine:** Ultimo commit del progetto;
- **Numero mesi slice:** 3 mesi.

Quindi, avremo che ogni slice sarà composta da 3 mesi di commit che vengono analizzati da Q-Spire per individuare gli smell al suo interno. Inoltre, la data d'inizio è significativa per due motivi principali:

- **Errori di versioning:** Dal 2020 è stata introdotta una nuova componente all'interno di Qiskit che ha cambiato la gestione di alcuni oggetti all'interno del framework. Infatti il tool va in conflitto con le versioni precedenti, rendendo impossibile le analisi;
- **Supremazia quantistica:** Da fine 2019 in poi, Google in primis e in seguito la Cina e altri paesi, hanno stabilito attraverso le loro ricerche che effettivamente il Quantum riesce a superare le prestazioni dei supercomputer attuali, dunque, da quel momento in poi, lo sviluppo dei progetti quantum ha preso sempre più piede nella community dei developer.

Queste regole sono state implementate all'interno di uno script, denominato `makeSlice.py`, che mi ha permesso di effettuare la divisione in maniera corretta, attraverso le GitHub API.

Lo script deve essere utilizzato in questo modo `python makeSlice.py {nomeRepo} -token {tokenGitHub}`.

Dopo ciò, basta riutilizzare lo script `counter.py` per calcolare le occorrenze degli smell individuati dal tool.

Tutti i risultati sono stati salvati all'interno del file `dataset_quantum_code_smells_sliced.xlsx`.

3.4 Utilizzo del Modello Statistico

Per le analisi dei dati che sono state applicate, è stato utilizzato il modello Cross-Sectional ben spiegato nella sezione 2.4.1.

Innanzitutto, abbiamo definito la *Prevalence*, cioè il rapporto di repository affette dallo smell e il numero totale di repository analizzate. Formalmente viene definita in questo modo:

$$P(X) = \frac{\text{Number of repositories affected by smell } X}{\text{Number of all repositories}}$$

In questo modo si riuscirà dunque a definire quale tra gli smells risulta essere statisticamente più presente all'interno delle repository.

L'altra analisi che è stata effettuata è sulla *Prevalence Odds Ratio (POR)*, che misura l'associazione tra la presenza e l'assenza di un *Code Smell* rispetto ad un altro. Per fare ciò, ho dovuto trasformare i dataset dei risultati da valori continui a binari, in modo tale che si potesse applicare il calcolo degli *Odds Ratio*. Poi, è bastato applicare la seguente formula:

$$POR = \frac{AD}{BC}$$

In questo modo riusciremo a capire come sono associate le coppie di *Code Smells*.

L'analisi verrà effettuata sia per il dataset dei risultati sull'ultima slice temporale (*dataset_quantum_code_smells.xlsx*) sia il dataset delle slice temporali, per capire l'evoluzione delle correlazioni nel tempo, scegliendo una serie di window dall'inizio del progetto.

Per quanto riguarda la prima analisi (sul dataset dell'ultima slice temporale), è stato ideato uno script, denominato *correlationDetection.py*, che, preso in input il dataset trasformato, mi ha permesso di ottenere in output due grafici:

- **Bar plot per il calcolo della prevalence**
- **Correlation matrix per la Prevalence Odds Ratio**

Lo script permette quindi di ottenere la *Prevalence* e la *POR* applicando le formule viste in precedenza all'interno del codice, per ottenere i risultati corretti.

Per quanto riguarda la seconda analisi (sul dataset delle slice temporali), ho innanzitutto preso un numero di window che fossero adatte per effettuare un'analisi consistente dei progetti. Infatti, il numero di window scelte è 7, avendo come punto di riferimento la data di inizio delle slice, così che si potesse capire da quel punto come si evolvessero i progetti in esame. Ciò ha prodotto un dataset di 25 progetti dai 40 che avevo a disposizione (denominato *dataset_quantum_code_smells_sliced_per_analysis.xlsx*), poiché, in alcuni casi, il numero di slice temporali era troppo piccolo per il limite fissato.

Fatto ciò, ho riapplicato le formule viste in precedenza, ma, in questo caso, la visualizzazione grafica è stata realizzata con un nuovo script, denominato *correlationSlicedDetection.py*, che permette di visualizzare 3 grafici differenti:

- **Heatmap della Prevalence;**
- **Heatmap della POR;**
- **Mini line-chart per coppia.**

Anche in questo caso sono state applicate le stesse formule di *Prevalence* e *POR*, ma divise per slice temporale (o window).

Capitolo 4

Risultati

4.1 Costruzione del Dataset ed Esecuzione del Tool

L'esecuzione del tool sui progetti ha portato ad escluderne alcuni presi in considerazione in precedenza, nella fase di costruzione del dataset, poiché il tool stesso non ha riscontrato risultati all'interno di essi.

Dunque, degli 88 progetti che ho analizzato, il tool ha dato risultati su 40 progetti che sono identificati con un'altra colonna all'interno del medesimo dataset, denominata *Results*, che ha i seguenti valori:

- **0**: Il tool non ha trovato alcun smell all'interno degli script del progetto;
- **1**: Il tool ha trovato smell all'interno degli script del progetto.

Questa distinzione è stata fondamentale per evitare di incorrere in errori nelle analisi che venivano effettuate. Di seguito viene mostrata la composizione del dataset dei progetti:

Tabella 4.1: Esempio righe *dataset_quantum-enabled_projects.xlsx*

URL	Flag	Results
https://github.com/qiskit-community/qiskit-machine-learning	1	1
https://github.com/Qiskit/qiskit-ibm-provider	1	0
https://github.com/mrtkp9993/QuantumComputingExamples	0	0

Si ricorda che la variabile *Flag* permette di distinguere quali progetti erano stati scelti rispetto alle caratteristiche illustrate nella sezione 3.2.

L'esecuzione di Q-Spire restituisce come risultato dell'analisi un semplice messaggio *Analysis Completed*, dove i risultati vengono raccolti in file .csv, con all'interno tutte le informazioni fondamentali sugli smells che vengono identificati per ogni script. Vediamo un esempio di risultato di output del tool:

Figura 4.1: *Esempio risultati Q-Spire*

circuit_name	column_end	column_start	explanation	matrix	qubits	row	suggestion	type
new_circ	22	21	,<Call to to_other_language>,,	1252	CG			
qiskit_circuit	79	41	,,,1593,,	LPQ				
qiskit_circuit	71	33	,,,1628,,	LPQ				

Tra le varie informazioni, la più importante che serve per il mio lavoro di tesi è la colonna *type*, che conserva al suo interno l'informazione riguardante il tipo di smell, il quale è stato riscontrato all'interno del singolo script.

In seguito, con l'utilizzo dello script *counter.py* (descritto sempre nella sezione 3.2), sono riuscito a raggruppare i dati per progetto. Vediamo un esempio di output dello script *counter.py*:

Figura 4.2: Esempio risultati script *counter.py*

RISULTATI DETTAGLIATI								
File	CG	ROC	NC	LC	IM	IdQ	IQ	LPQ
_mpqp_core_circuit.py.csv	1	0	0	0	0	0	0	2
_mpqp_execution_providers_ibm.	0	0	0	0	0	0	0	1
_mpqp_tools_circuit.py.csv	0	0	0	0	0	0	0	2
TOTALE	1	0	0	0	0	0	0	5

File Excel salvato come: results\mpqp\results.xlsx

Grazie allo script *counter.py*, sono riuscito ad ottenere in maniera rapida ed efficiente i risultati sia divisi per script, salvandoli nel file *results.xlsx*, sia il totale del progetto che poi ho inserito come label nel dataset dei risultati totali, che vedremo successivamente. Vediamo un esempio di *results.xlsx*:

Figura 4.3: *results.xlsx*

Script	CG	ROC	NC	LC	IM	IdQ	IQ	LPQ
\mpqp\core\circuit.py	1	0	0	0	0	0	0	2
\mpqp\execution\providers\ibm.py	0	0	0	0	0	0	0	1
\mpqp\tools\circuit.py	0	0	0	0	0	0	0	2

Come si può notare dall'immagine, i risultati sono divisi per script e ogni script ha i suoi smell identificati dall'acronimo. Ogni colonna può assumere due valori:

- **0**: Lo smell non è presente all'interno dello script;
- **> 0**: Lo smell è presente all'interno dello script.

Infine, i risultati singoli sono stati raggruppati e salvati all'interno di un dataset, denominato *dataset_quantum_code_smells.xlsx*, dove sono presenti tutti i progetti, con tutte le occorrenze sommate dei singoli script degli smells identificati. Vediamo un esempio delle righe del dataset:

Figura 4.4: *dataset_quantum_code_smells.xlsx*

Repo	CG	ROC	NC	LC	IM	IdQ	IQ	LPQ
isd-quantum	0	0	0	0	1	13	3	0
kaleidoscope	0	1	0	0	0	3	1	0
mpqp	1	0	0	0	0	0	0	5
piQture	0	10	0	0	2	62	4	0
plank-quantum	0	8	0	0	0	16	5	0

4.2 Divisione in Slice Temporali

La seconda parte del mio lavoro di tesi mi ha permesso di dividere i progetti presi in considerazione in slice temporali con le caratteristiche descritte nella sezione 3.3. Questa divisione è stata fondamentale per capire l'evoluzione dei progetti Quantum.

Per fare ciò, ho dovuto eseguire su tutti i progetti lo script *makeSlice.py*, che mi ha restituito in output la divisione in più cartelle del progetto che veniva passato da terminale. Infatti, lo script va ad utilizzare le GitHub API per navigare nella cronologia dei commit e prelevare le informazioni salvate all'interno di un intervallo di tempo di 3 mesi, finché non si arriva all'ultimo commit, che risulta essere quello temporalmente più recente.

Lo script, nel caso in cui non trovi alcun commit all'interno di un intervallo temporale, effettua uno skip all'intervallo successivo, finché non trovi almeno un nuovo commit. Ciò ha permesso di evitare di effettuare salvataggi ridondanti e inutili.

Successivamente, è stato eseguito il tool su tutti gli intervalli (chiamati anche *snapshots*) di tutti i 40 progetti presi in esame, allo stesso modo di come era stato fatto per i risultati della sezione 4.1. L'unica differenza la si può riscontrare nel dataset finale, denominato *dataset_quantum_code_smells_sliced.xlsx*, che raggruppa i totali dei progetti, poiché, in questo caso, i risultati sono raggruppati per repository e poi divisi nelle fasce temporali riscontrate dallo script *makeSlice.py*. Vediamo come si presentano i risultati finali per questa fase:

Figura 4.5: *dataset_quantum_code_smells_sliced.xlsx*

REPO									
mpqp									
#	Anno	CG	ROC	NC	LC	IM	IdQ	IQ	LPQ
10	2023-12-11_to_2024-03-10	1	1	0	0	0	0	0	1
11	2024-03-10_to_2024-06-08	1	0	0	0	0	0	0	1
12	2024-06-08_to_2024-09-06	3	2	0	0	0	0	0	3
13	2024-09-06_to_2024-12-05	2	0	0	0	0	0	0	3
14	2024-12-05_to_2025-03-05	2	0	0	0	0	0	0	3
15	2025-03-05_to_2025-06-03	1	0	0	0	0	0	0	4
16	2025-06-03_to_2025-08-25	1	0	0	0	0	0	0	5
REPO									
piQture									
#	Anno	CG	ROC	NC	LC	IM	IdQ	IQ	LPQ
17	2022-09-17_to_2022-12-16	0	0	0	0	0	0	0	0
18	2022-12-16_to_2023-03-16	0	2	0	0	0	2	0	0
19	2023-03-16_to_2023-06-14	0	2	0	0	0	2	0	0
20	2023-06-14_to_2023-09-12	0	0	0	0	0	4	0	0

Questi risultati sono stati ottenuti utilizzando sempre lo script *counter.py* sulle slice derivate. Una cosa interessante che ci ha fornito questo lavoro, è che anche i software quantum-enabled si comportano come i software tradizionali, dove più aumenta lo scope del progetto e più vi è l'insorgere dei "problemi" all'interno del software. Infatti, l'andamento solito dei dati per le slice temporali è il seguente: *all'aumentare del tempo impiegato sul progetto, aumentano gli smell individuati all'interno dei progetti*.

Ciò può essere spiegato dall'aumentare del numero di script all'interno dei progetti nel corso del tempo e dal numero di persone che collaborano all'interno di questi progetti open-source Quantum.

4.3 Utilizzo del Modello Statistico

L'ultima parte del lavoro di tesi mi ha consentito di analizzare in maniera dettagliata i progetti open-source e di rispondere alle domande di ricerca che mi ero posto.

4.3.1 Prevalence

Per la *Prevalence* riporto le domande di ricerca inerenti a questo argomento:

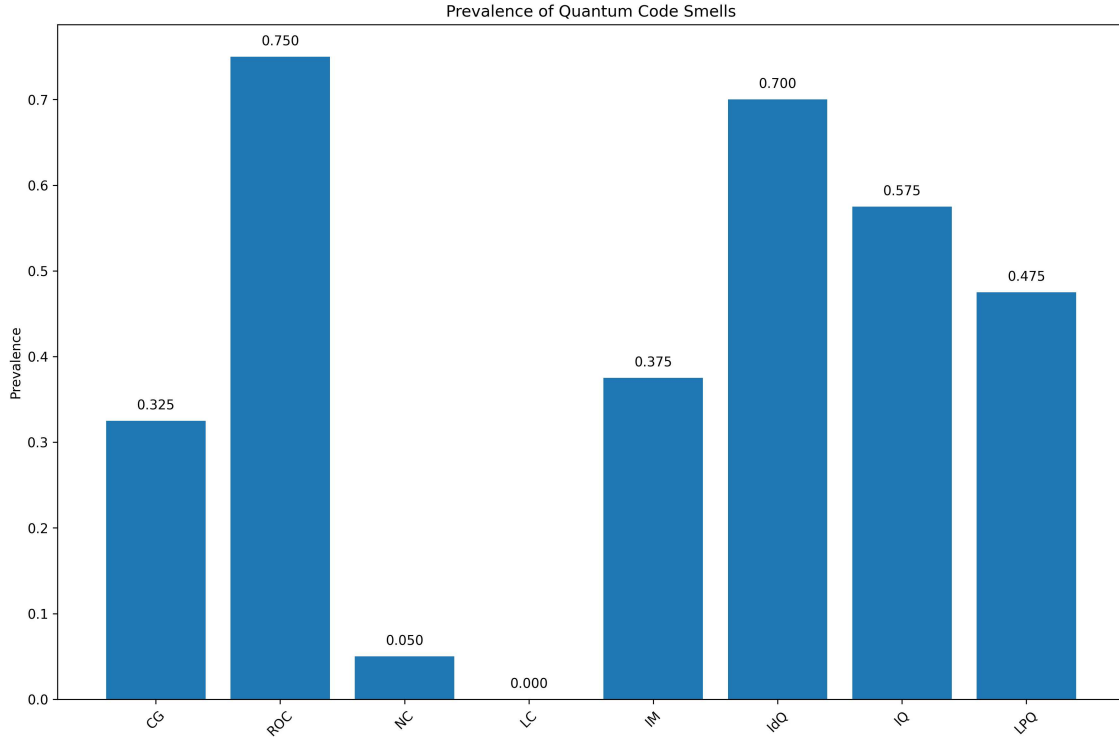
Domande di ricerca

RQ1: Qual è la *Prevalence* dei *Code Smells* nei progetti Quantum?

RQ2: Come varia nel tempo la *Prevalence* dei *Code Smells* in progetti Quantum?

Per quanto riguarda la *RQ1*, ho dunque calcolato la *Prevalence* per ogni progetto come mostrato nella sezione 3.4, ciò mi ha fornito i seguenti risultati:

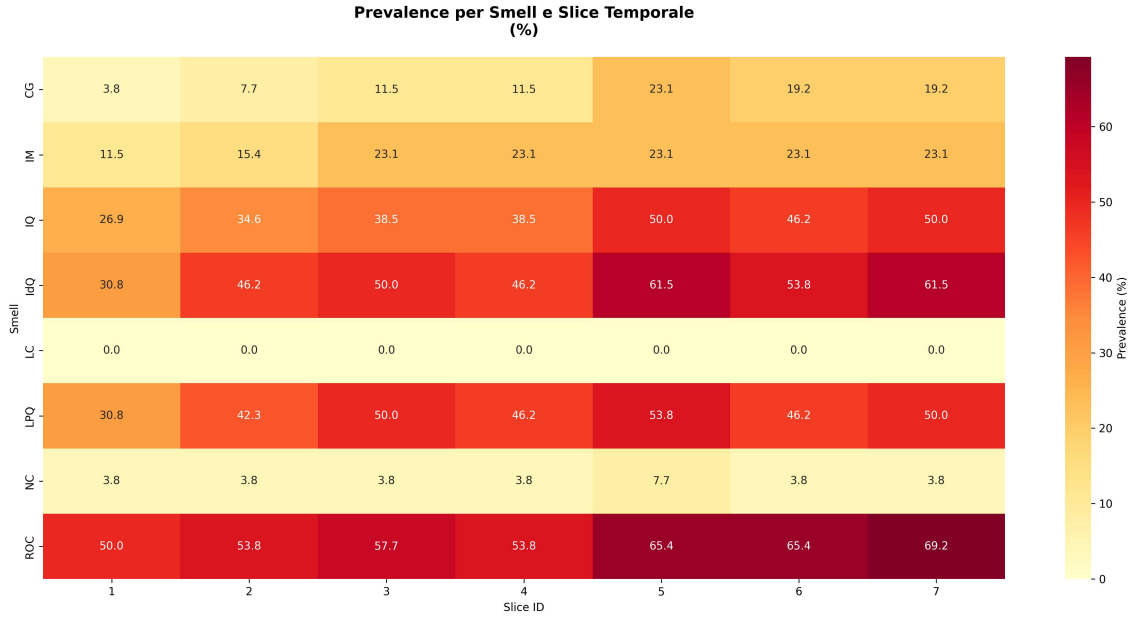
Figura 4.6: *Prevalence dei Quantum Code Smells*



Come si può notare dalla figura 4.6, lo smell più presente all'interno dei progetti presi in esame è *ROC* (*Repeated set of Operations on Circuit*) con il 75% di progetti in cui è presente questo smell, seguito da *IdQ* (*Idle Qubits*) con il 70% e poi vi è *IQ* (*Initialization of Qubits differently from $|0\rangle$*) con il 57,5%. Gli altri smells si assestano sotto il 50%, con un'assenza molto rilevante di *LC* (*Long Circuits*) e *NC* (*Non-parameterized Circuits*), rispettivamente con 0% e 0,05%. Quindi nei progetti che sono stati analizzati, non sono mai state rilevate occorrenze di *LC* e 3/4 dei progetti soffrono di *ROC*.

Per la *RQ2*, ho prodotto una heatmap dove ogni riga rappresenta l'evoluzione dello smell e ogni colonna rappresenta le finestre temporali analizzate. Ricordo che le finestre temporali sono 7, che vanno dal 2020-01-01, con una differenza temporale tra le slice di 3 mesi. Di seguito mostro i risultati:

Figura 4.7: *Trends della Prevalence dei Quantum Code Smells*



Da una prima analisi dei risultati nella figura 4.7, si può notare come il colore tende a crescere di intensità e di grado, ciò ci può far capire che, in linea generale, con l'aumentare del tempo aumenta la *Prevalence* di uno specifico smell all'interno dei progetti Quantum. Ciò molto probabilmente è dovuto all'aumento della complessità dei progetti nel corso del tempo.

Andando nel dettaglio, si conferma *ROC* lo smell più presente all'interno dei progetti Quantum. Una cosa interessante che ho notato è che in alcune window temporali vi è maggiore presenza di *LPQ*, rispetto alla *Prevalence* totale. Inoltre, l'andamento non è proprio una crescita costante, ma in alcuni momenti ci sono dei leggeri cali, molto probabilmente dovuti a piccoli aggiornamenti che hanno migliorato il codice e risolto alcune problematiche, come nel caso di *ROC*, tra le slice 3 e 4.

4.3.2 POR

Anche per la *POR* riporto le domande di ricerca inerenti a questo argomento:

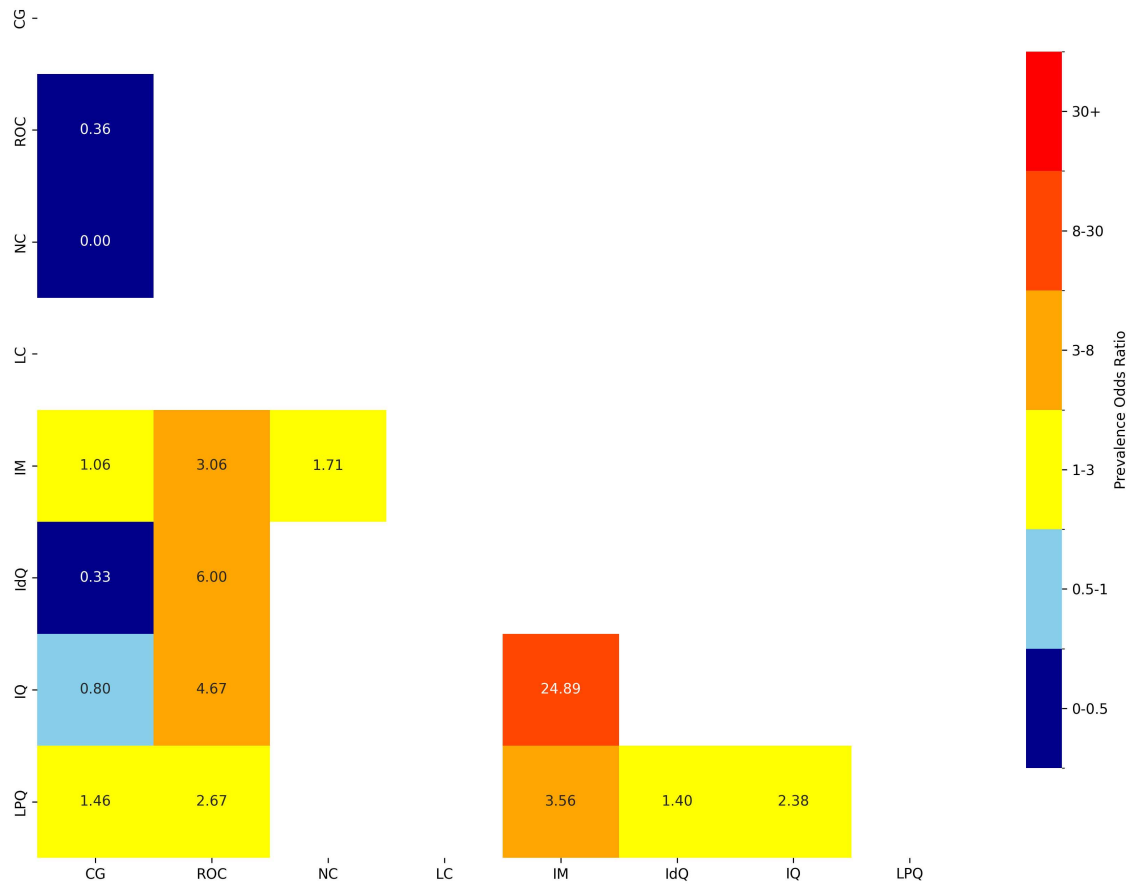
Domande di ricerca

- RQ3:** Quali sono le relazioni tra differenti Code Smells in progetti Quantum?
RQ4: Come variano nel tempo le relazioni tra differenti Code Smells in progetti Quantum?

In merito alla *RQ3*, ho calcolato la *POR* (*Prevalence Odds Ratio*), per ogni coppia di smells e prodotto una matrice delle correlazioni. Di seguito i risultati riscontrati:

Figura 4.8: *POR dei Quantum Code Smells*

Prevalence Odds Ratio Matrix
(Custom Clinical Scale: 0-0.5-1-3-8-30+)



Per rendere i risultati più chiari e leggibili sono state create delle fasce per stabilire la forza delle associazioni:

- **Blu scuro:** Rappresenta una forte associazione negativa tra i due Code Smells;
- **Azzurro:** Rappresenta una debole associazione negativa tra i due Code Smells;
- **Giallo:** Rappresenta una debole associazione positiva tra i due Code Smells;
- **Arancione:** Rappresenta una moderata associazione positiva tra i due Code Smells;
- **Rosso-arancio:** Rappresenta una forte associazione positiva tra i due Code Smells;
- **Rosso:** Rappresenta un'associazione molto forte positiva tra i due Code Smells.

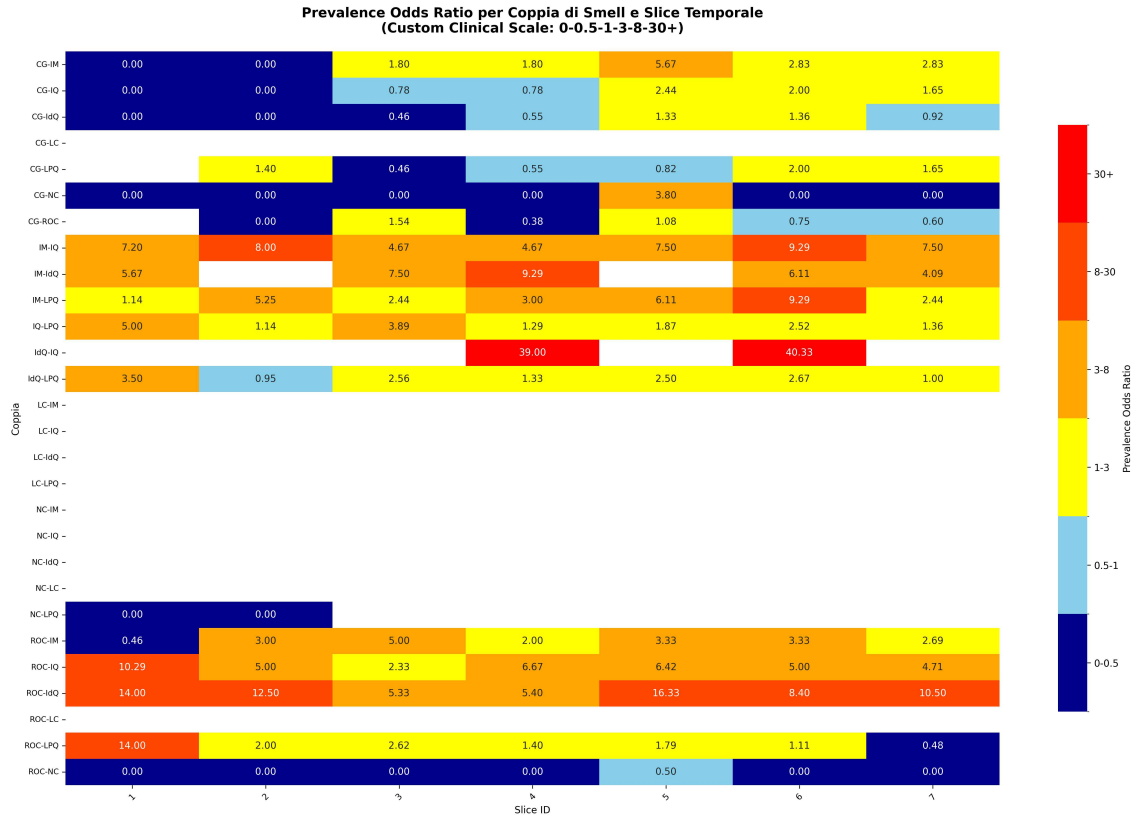
Andando ad analizzare nel dettaglio la matrice prodotta, oltre ad alcuni campi vuoti che rappresentano valori NaN, generati dalla divisione per zero nel rapporto della *POR*, Ci sono risultati molto interessanti:

- **Coppia NC-CG:** La *POR* è uguale a 0,00, ciò indica una associazione negativa perfetta, quindi potrebbe voler dire che i due smell si tendono ad escludere a vicenda sempre;
- **Coppia ROC-CG:** La *POR* è uguale a 0,36, dunque si ha una associazione negativa forte che indica probabilmente un esclusione a vicenda dei due Code Smells;
- **Coppia IdQ-CG:** La *POR* è uguale a 0,33, anche in questo caso si ha una associazione negativa forte;
- **Coppia IM-ROC:** La *POR* è uguale a 3,06, in questo caso si ha una associazione positiva moderata, ciò potrebbe voler dire che i due smell tendono a presentarsi insieme;
- **Coppia IdQ-ROC:** La *POR* è uguale a 3,56, anche in questo caso si ha una associazione positiva moderata;
- **Coppia IQ-ROC:** La *POR* è uguale a 4,67, dunque si ha una associazione positiva moderata;
- **Coppia LPQ-IM:** La *POR* è uguale a 6,00, un'associazione che tende ad essere più forte delle precedenti, ma sempre classificata come moderata.
- **Coppia IQ-IM:** La *POR* è uguale a 24,89, in questo caso si ha un'associazione positiva forte, dunque gli smell tendono sempre di più a presentarsi insieme ed indicare che probabilmente uno causa l'altro.

La coppia sicuramente più interessante ed evidente da analizzare è la coppia *IQ-IM*, che ha una forte associazione positiva. Andando a rivedere le definizioni tecniche di IQ e IM, descritte nella tabella 2.1. Questa associazione può essere spiegata da una dipendenza tecnica reale, dovuta al fatto che un programmatore quantistico eseguendo misurazioni intermedie (IM), altera lo stato dei qubit e porta a reinizializzare i qubit in stati diversi rispetto allo stato standard $|0\rangle$ (IQ).

Per quanto riguarda la *RQ4*, ho prodotto una heatmap che mostra come si evolve la *POR* nel tempo, utilizzando sempre le stesse window temporali, utilizzate per rispondere alla *RQ2*. Di seguito mostro i risultati:

Figura 4.9: Trends della POR dei Quantum Code Smells

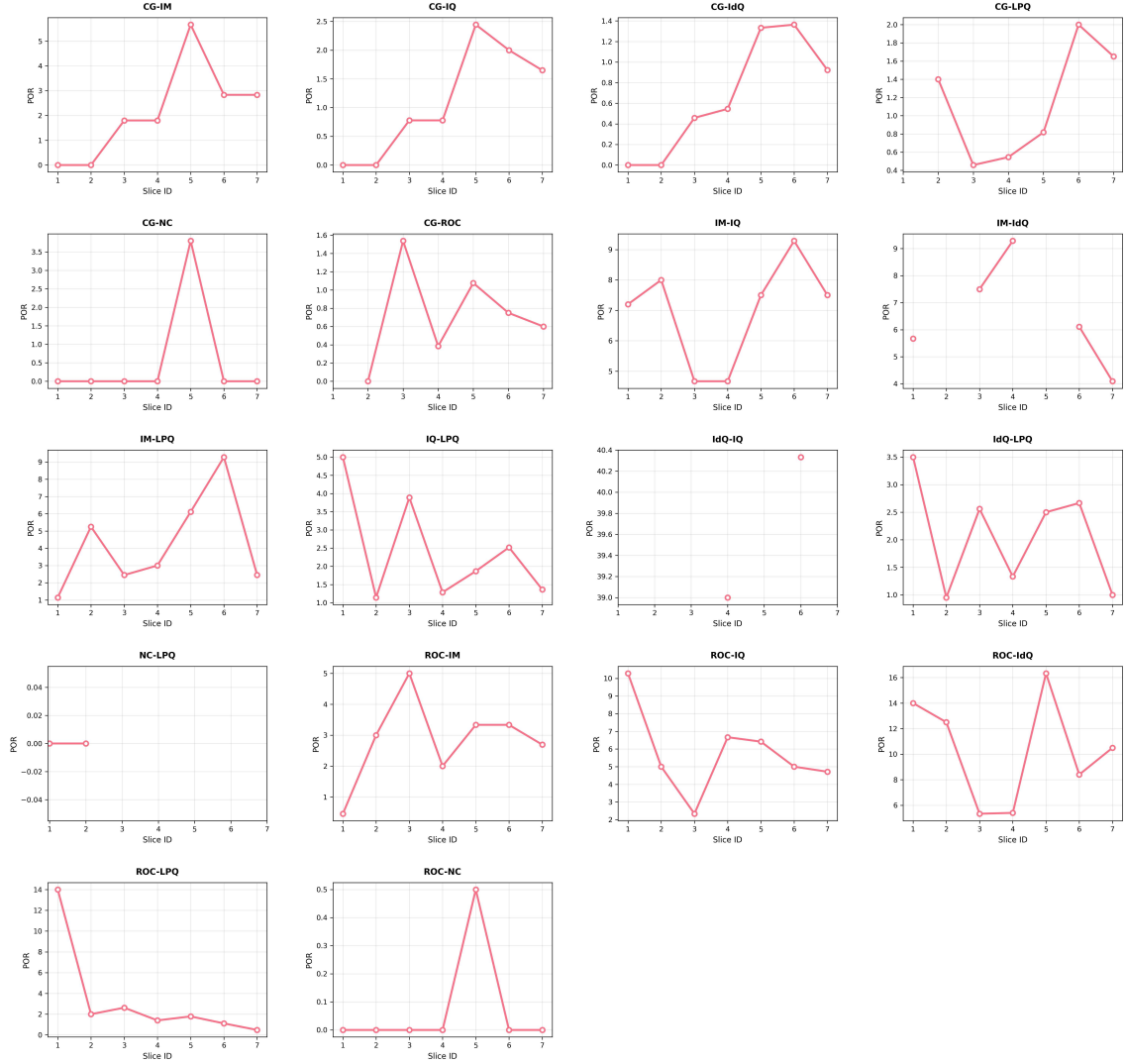


Confrontando i risultati ottenuti sull'ultima slice temporale con i risultati che ho appena mostrato, si può notare che in linea generale vi è un aumento della *POR* (infatti si passa da colori freddi a colori caldi) e si rispettano i valori che erano stati riscontrati sulla risposta alla *RQ3*. Solo in alcuni casi vi è un comportamento "anomalo" (come per esempio la coppia ROC-LPQ), in cui la curva di crescita del calore è sinusoidale, quindi ci sono picchi di punti "caldi" che si alternano a picchi freddi, oppure risulta essere totalmente discendente.

Per evidenziare ancora di più i trends dei valori di *POR* nel corso del tempo, ho prodotto un altro grafico che permette di vedere in un multi-lines l'andamento della *POR*, per ogni coppia di smell che assume almeno un valore non NaN, di seguito mostro i risultati:

Figura 4.10: *Trends della POR dei Quantum Code Smells*

POR Trends per Coppia di Smell



Anche in questa figura si può notare come tendenzialmente i risultati dalla prima window, iniziano a crescere e poi ci sono picchi più "freddi" che si alternano a picchi più "caldi", non c'è una vera e propria crescita continua. Ciò può essere dovuto dal fatto che, gli script che avevano quel tipo di problemi sono stati rimossi oppure modificati e ciò ha portato ad una riduzione temporanea di valori che si è ripresentata nelle slice successive.

Capitolo 5

Conclusioni

5.1 Riassunto dei Contenuti

In questa tesi sono riuscito ad acquisire conoscenze importanti sul Quantum Computing e su quelli che sono i Quantum Code Smells. Oltre ad aver evidenziato, per ogni progetto, gli smells che sono stati rilevati attraverso Q-Spire, ho effettuato un'analisi di tipo cross-sectional per evidenziare la *Prevalence* e la *POR* all'interno dei progetti, per capire quali fossero gli smells più presenti e se ci fosse qualche tipo di dipendenza tra gli smells. Inoltre, ho voluto condurre uno studio temporale dei progetti, per evidenziare eventuali pattern comuni nel tempo, attraverso la suddivisione dei progetti in slice (o window) temporali.

Tutto ciò è stato fatto tramite la raccolta dei progetti da GitHub, in un dataset specifico, la detection degli smells dei progetti attraverso Q-Spire e l'analisi statistica sui risultati, svolta attraverso degli script in Python. I risultati sono stati evidenziati in maniera grafica, attraverso vari plot (bar plot, ecc.).

I risultati sono stati molto soddisfacenti, poiché hanno permesso di mostrare l'esistenza di alcune associazioni importanti, tra cui, spicca quella tra *IQ* (*Initialization of Qubits differently from $|0\rangle$*) e *IM* (*Intermediate Measurements*), con un'associazione positiva di valore 24,89.

5.2 Lavori Futuri

Considerando il lavoro sperimentale che è stato effettuato, si possono sicuramente apportare delle migliorie sia a Q-Spire che alle analisi effettuate. Vediamo un elenco di possibili lavori futuri che possono essere svolti sulla base di questa tesi:

- **Allargare il numero di progetti:** In questo momento il numero di progetti è consistente per il tipo di analisi svolta, ma si potrebbe rendere il tutto ancora più consistente e importante con l'utilizzo di un numero di progetti ancora più grande. In questo momento, siamo ancora agli inizi e non è semplice trovare progetti con caratteristiche adatte, però in futuro, con un'eventuale "esplosione" del quantum, può essere più facile ed utile effettuare questo lavoro.
- **Migliorie a Q-Spire:** Alfieri ha già svolto un ottimo lavoro nella sua tesi. Sviluppare un tool di detection di questo tipo da zero non è semplice, però si potrebbero applicare alcuni miglioramenti che potrebbero rendere la detection meno pesante e più efficiente. Per esempio, su alcuni progetti, Q-Spire impiega molto tempo ad arrivare ai risultati, quindi si potrebbe pensare di ottimizzare il tool per evitare queste perdite di tempo.

- **Effettuare ulteriori analisi statistiche:** Si potrebbe pensare di allargare l'analisi statistica, andando ad effettuare un'analisi descrittiva dei vari smells oppure fornire ulteriori plot di visualizzazione dei risultati.
- **Utilizzare progetti privati:** Si potrebbe pensare di utilizzare dei progetti differenti da quelli open-source, nei quali potrebbero essere individuati dei pattern nascosti che consentirebbero di ottenere ulteriori risultati importanti.

Bibliografia

- [1] ai4business. *Mercato dei quantum computer a quota 500 mld di euro nel 2030*. <https://www.ai4business.it/quantum-computing/mercato-dei-quantum-computer-a-quota-500-mld-di-euro-nel-2030/>. 2024.
- [2] Omar Al Omari et al. “Prevalence and predictors of depression, anxiety, and stress among youth at the time of COVID-19: an online cross-sectional multicountry study”. In: *Depression research and treatment* (2020). DOI: <https://doi.org/10.1155/2020/8887727>.
- [3] Francesca Arcelli Fontana et al. “Comparing and experimenting machine learning techniques for code smell detection”. In: *Empirical Software Engineering* 21 (2016), pp. 1143–1191.
- [4] Roberta Arcoverde, Alessandro Garcia, and Eduardo Figueiredo. “Understanding the longevity of code smells: preliminary results of an explanatory survey”. In: *Proceedings of the 4th Workshop on Refactoring Tools*. 2011, pp. 33–36.
- [5] Paris Avgeriou et al. “Managing technical debt in software engineering (dagstuhl seminar 16162)”. In: *Dagstuhl reports* 6.4 (2016), pp. 110–138.
- [6] Paris C Avgeriou et al. “An overview and comparison of technical debt measurement tools”. In: *Ieee software* 38.3 (2020), pp. 61–71.
- [7] Maurizio Carpita. *La qualità del lavoro nelle cooperative sociali: misure e modelli statistici*. Franco Angeli, 2009.
- [8] Alexander Chatzigeorgiou and Anastasios Manakos. “Investigating the evolution of bad smells in object-oriented code”. In: *2010 Seventh International Conference on the Quality of Information and Communications Technology*. IEEE. 2010, pp. 106–115.
- [9] Qihong Chen et al. “The smelly eight: An empirical study on the prevalence of code smells in quantum computing”. In: *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE. 2023, pp. 358–370.
- [10] Ward Cunningham. “The WyCash portfolio management system”. In: *ACM Sigplan Ops Messenger* 4.2 (1992), pp. 29–30.
- [11] Manuel De Stefano et al. “The quantum frontier of software engineering: A systematic mapping study”. In: *Information and Software Technology* (2024), p. 107525.
- [12] Dario Di Nucci et al. “Detecting code smells using machine learning techniques: Are we there yet?” In: *2018 iee 25th international conference on software analysis, evolution and reengineering (saner)*. IEEE. 2018, pp. 612–621.
- [13] Marcello Gallucci, Luigi Leone, et al. *Modelli statistici per le scienze sociali*. Pearson Education, 2012.

- [14] Google. *Cirq: A Python Framework for Creating, Editing, and Invoking Noisy Intermediate Scale Quantum Circuits*. URL: <https://quantumai.google/cirq>.
- [15] IBM. *IBM Quantum Platform*. URL: <https://quantum-computing.ibm.com>.
- [16] Heng Li, Foutse Khomh, Moses Openja, et al. “Understanding quantum software engineering challenges an empirical study on stack exchange forums and github issues”. In: *2021 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE. 2021, pp. 343–354.
- [17] Tao Lin et al. “A novel approach for code smells detection based on deep leaning”. In: *Applied Cryptography in Computer and Communications: First EAI International Conference, AC3 2021, Virtual Event, May 15-16, 2021, Proceedings 1*. Springer. 2021, pp. 171–174.
- [18] Hui Liu et al. “Deep learning based code smell detection”. In: *IEEE transactions on Software Engineering* 47.9 (2019), pp. 1811–1837.
- [19] Ruchika Malhotra, Bhawna Jain, and Marouane Kessentini. “QNet: exploring deep learning for quantum code smell detection”. In: *Software Quality Journal* 33.2 (2025), pp. 1–28.
- [20] Antonio Martini, Jan Bosch, and Michel Chaudron. “Investigating architectural technical debt accumulation and refactoring over time: A multiple-case study”. In: *Information and Software Technology* 67 (2015), pp. 237–253.
- [21] Patrick Masiero. “Ambiente per l’analisi della qualita’dei software open source”. In: ().
- [22] Microsoft. *Q# Programming Language*. URL: <https://learn.microsoft.com/en-us/azure/quantum/user-guide/language/>.
- [23] Enrique Moguel et al. “A Roadmap for Quantum Software Engineering: Applying the Lessons Learned from the Classics.” In: *Q-SET@ QCE*. 2020, pp. 5–13.
- [24] Naouel Moha et al. “Decor: A method for the specification and detection of code and design smells”. In: *IEEE Transactions on Software Engineering* 36.1 (2009), pp. 20–36.
- [25] Michael A Nielsen and Isaac L Chuang. *Quantum computation and quantum information*. 1991.
- [26] Moses Openja et al. “Technical debts and faults in open-source quantum software systems: An empirical study”. In: *Journal of Systems and Software* 193 (2022), p. 111458.
- [27] Fabio Palomba et al. “A large-scale empirical study on the lifecycle of code smell co-occurrences”. In: *Information and Software Technology* 99 (2018), pp. 1–10.
- [28] Fabio Palomba et al. “A textual-based technique for smell detection”. In: *2016 IEEE 24th international conference on program comprehension (ICPC)*. IEEE. 2016, pp. 1–10.
- [29] Fabio Palomba et al. “Mining version histories for detecting code smells”. In: *IEEE Transactions on Software Engineering* 41.5 (2014), pp. 462–489.
- [30] Fabiano Pecorelli et al. “A large empirical assessment of the role of data balancing in machine-learning-based code smell detection”. In: *Journal of Systems and Software* 169 (2020), p. 110693.

- [31] Fabiano Pecorelli et al. “Comparing heuristic and machine learning approaches for metric-based code smell detection”. In: *2019 IEEE/ACM 27th international conference on program comprehension (ICPC)*. IEEE. 2019, pp. 93–104.
- [32] Fabiano Pecorelli et al. “Developer-driven code smell prioritization”. In: *Proceedings of the 17th International Conference on Mining Software Repositories*. 2020, pp. 220–231.
- [33] Fabiano Pecorelli et al. “On the adequacy of static analysis warnings with respect to code smell prediction”. In: *Empirical Software Engineering* 27.3 (2022), p. 64.
- [34] Mario Piattini et al. “The Talavera Manifesto for quantum software engineering and programming.” In: *QANSWER*. 2020, pp. 1–5.
- [35] Qiskit Development Team. *Qiskit: An Open-source Framework for Quantum Computing*. URL: <https://qiskit.org>.
- [36] Saurabh R Shrivastava and Prateek S Bobhate. “Computer related health problems among software professionals in Mumbai- A cross-sectional study”. In: *Safety Science Monitor* 15 (2012), pp. 1–6.
- [37] Julia Simkus. “Cross-Sectional Study: Definition, Designs & Examples”. In: (2023). URL: <https://www.simplypsychology.org/what-is-a-cross-sectional-study.html>.
- [38] Lauren Thomas. “Cross-Sectional Study — Definition, Uses & Examples”. In: (2020). URL: <https://www.scribbr.com/methodology/cross-sectional-study/#:~:text=A%20cross%2Dsectional%20study%20is,observe%20variables%20without%20influencing%20them>.
- [39] Nikolaos Tsantalis and Alexander Chatzigeorgiou. “Identification of move method refactoring opportunities”. In: *IEEE Transactions on Software Engineering* 35.3 (2009), pp. 347–367.
- [40] Michele Tufano et al. “An empirical investigation into the nature of test smells”. In: *Proceedings of the 31st IEEE/ACM international conference on automated software engineering*. 2016, pp. 4–15.
- [41] Michele Tufano et al. “When and why your code starts to smell bad”. In: *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. Vol. 1. IEEE. 2015, pp. 403–414.
- [42] Jianjun Zhao. “Quantum software engineering: Landscapes and horizons”. In: *arXiv preprint arXiv:2007.07047* (2020).

Ringraziamenti

Sono passati altri due anni da quello che è stato il mio primo traguardo universitario e sembrano essere volati in un battito di ciglia. Ho arricchito ancora di più il mio bagaglio culturale e ho conosciuto nuove persone speciali, che mi hanno regalato tante emozioni, sia positive che negative. Anche in questo caso, come per la triennale, le difficoltà sono state tante e quando è così c'è sempre quel piccolo dubbio che ti può portare fuori strada, ma, spinto dalla voglia di imparare e di migliorare, si può superare anche l'impossibile. So che la strada è ancora lunga e che ci saranno tante altre difficoltà e ostacoli da affrontare, ma so anche che più è difficile il viaggio, più sarà bello arrivare al traguardo. Non nego di avere paura di ciò che mi aspetta, io cerco di mostrarmi sicuro di ciò che faccio, la verità è che sono un essere umano e come tale ho le mie paure e le mie debolezze, però, so che ho tante persone affianco a me che mi vogliono bene e che credono in me e questo mi dà la forza per andare avanti e di non mollare mai, rendendo le paure più facili da affrontare. Mi mancherà tanto Fisciano e forse con alcune persone non riusciremo a vederci come al solito, però questo non vuol dire che il legame scomparirà, io sarò sempre pronto e disponibile a dare una mano e essere presente per voi.

Vorrei ringraziare mia madre, una Donna con la D maiuscola, bella e intelligente, si fa in quattro per noi senza chiedere nulla in cambio, io cerco di aiutarla ma lei non ne vuole sapere, anche perché è difficile stare al suo passo. Spero che tu sia fiera di quello che sono e ciò che ho fatto fino ad ora e anche se magari da ora in avanti ci potrà essere più distanza, per lavoro o per altre situazioni, sappi che io sono sempre qui in qualsiasi momento e con gli strumenti di oggi anche le distanze possono essere minimizzate. Ti voglio tanto bene.

Ringrazio mio padre, un Uomo con la U maiuscola, spesso ti prendi carico di cose che non dovresti, però è anche vero che questo è il tuo ruolo in questa storia che stiamo scrivendo insieme. Sappi che per quanto sia difficile e dura da digerire ciò che hai vissuto, noi saremo sempre al tuo fianco e qualunque scelta tu prenda saremo lì a supportarti nelle difficoltà. Questi sono i valori che ci hai insegnato e ciò ti può fare solo onore. A volte ci sono scelte che sono difficili da prendere e la scelta più logica sarebbe quella di condividere tutto ciò che hai dentro per alleggerire il carico, ma tu riesci a sopportare tutto ciò e ad andare avanti, ti ammiro e spero di essere un Uomo come te, anzi un superuomo. Ti voglio tanto bene.

Ringrazio mio fratello, hai una forza e una resilienza che in pochi ho visto, il fatto che ti fai in quattro per avere la tua indipendenza, anche sprecando tempo che potresti investire per un'uscita in più, ti fa onore e lo ammiro. Purtroppo la vita ci ha messo in situazioni particolari fin da bambino, però, anche grazie ai nostri genitori siamo riusciti ad andare avanti e a non farci influenzare da cattiveria e invidia. La cosa che mi preoccupava di più era che raggiungere questo traguardo prima di te avrebbe potuto portarti ansia e invidia, invece tu speravi che io finissi prima e questo è qualcosa che solo un fratello può fare. Sappi che ognuno ha i suoi tempi e non pensare alle persone e a ciò che possono dire o pensare, tu sei il mio

modello ed è grazie anche a te, se sono la persona che sono oggi. Vai per la tua strada, sempre oltre il limite e non smettere di sognare. Ti voglio tanto bene.

Ringrazio i ragazzi del gruppo "Club NEXT GEN", che mi hanno aiutato nel corso dell'ultimo anno a superare i momenti di noia e solitudine con una semplice chiamata di Discord, una serata di gaming tra di noi o anche una breve uscita. Siete persone fantastiche, ognuno con le proprie caratteristiche e spero che ci sia sempre un sano e spontaneo rapporto tra di noi e che tutto ciò vada oltre la distanza che si potrà creare nei prossimi anni.

Un ringraziamento speciale va a Lello e Cosimo, con i quali ho stretto un legame molto forte negli ultimi mesi, mi avete aiutato a superare le difficoltà e condividere momenti di svago. Siete persone incredibili, con le quali sto condividendo tutt'oggi le giornate e sappiate che vi meritate il meglio che la vita vi può dare. Non perdetevi la bontà che vi contraddistingue e puntate sempre al massimo.

Ringrazio i ragazzi del gruppo "AURENGERS", con i quali ci siamo incrociati di meno, ma quando ci incontriamo è sempre una festa, abbiamo creato un rapporto negli ultimi anni che va oltre la distanza. Continuate ad inseguire i vostri sogni e non mollate mai, ognuno ha i suoi tempi per fare le cose e non vi preoccupate del giudizio altrui, spero che raggiungete i vostri sogni.

Un ringraziamento speciale va a Luigi, il mio migliore amico, con te ho condiviso di tutto, anche le mie passioni, i miei desideri, le mie paure. Sappi che ci sarò sempre per qualsiasi cosa. Sei una persona forte e gentile che ha dei principi invidiabili. Ricorda che "I sogni delle persone non avranno mai fine".

Ringrazio i ragazzi del gruppo "Skibidi boppy forza napoli", siete stati dei compagni fantastici con i quali ho condiviso le lezioni e i seminari, mi avete aiutato a crescere e alleviare le giornate pesanti di università. Mantenete sempre la vostra leggerezza e affrontate ogni difficoltà con il sorriso.

Un ringraziamento speciale va a Marica, per le sfide a scacchi, per le giornate passate insieme a Fisciano a studiare, per le passeggiate rigeneranti e le chiacchierate distensive, per gli scambi di reel divertenti. La tua leggerezza e simpatia sono state preziose in questo intenso periodo, sei stata una compagna di viaggio unica. Inseguì i tuoi sogni e non mollare mai, io credo in te.

Ringrazio Daniele, con te ho condiviso questi due splendidi anni. Ci siamo sempre sfiorati ma mai conosciuti, poi c'è stata l'occasione di farlo (ricordo ancora quando non sapevi dove fare la discussione per la tesi triennale) e da lì siamo stati quasi sempre insieme, tra progetti, sogni e paure degli esami. Senza di te, sarebbe stata ancora più dura e difficile. So che il futuro un po' ti spaventa, ma so che sei preparato e il futuro che hai davanti è radioso.

Ringrazio Nicola, con te ho condiviso una delle sfide più difficili e importanti della mia vita, il "temutissimo" esame di Compilatori. Sei stato un compagno stupendo con cui mi sono divertito tantissimo. Mi mancano un po' quei momenti di pausa in cui ci facevamo il "gamedle" e cercavamo di riuscire ad indovinare tutti e quattro i quesiti. Continua ad inseguire i tuoi sogni e le tue passioni. Hai avuto difficoltà importanti sul tuo cammino, ma non mollare mai, vedrai che col tempo tutto si sistema. P.S. Miles Morales > Peter Parker.

Ringrazio il prof. Palomba, relatore per la mia tesi. Persona incredibile che cerca di portare innovazione e freschezza nelle lezioni. La ringrazio per la sua disponibilità e spero che continui a lavorare in questo modo. È stato fantastico condividere con lei questa tesi.

Ringrazio il dott. Lambiase, sei una persona fantastica e premurosa che riesce a trovare tempo per tutti i suoi tesisti. So che adesso stai svolgendo un lavoro all'estero e che magari questa cosa ti rende più distante, però sappi che è solo questione di tempo e abitudine. La tesi è stata molto più facile grazie a te e ai tuoi consigli e suggerimenti. Spero che tra qualche anno ti potrò chiamare professore, perché te lo meriti.

Ringrazio anche tutti i professori che ho incontrato nel corso degli anni. Ognuno di loro mi ha permesso di acquisire conoscenze e competenze fondamentali per il mio futuro.

Alla triennale non lo avevo fatto ma stavolta mi sento di farlo, ringrazio anche me stesso, per la mia caparbia, determinazione e voglia di imparare e studiare. Spingiti sempre oltre i limiti e non mollare mai, sei forte e lo stai dimostrando con i fatti. Non perdere mai la via che stai tracciando, sii gentile, premuroso, aiuta sempre il prossimo e cerca di non perdere mai quell'empatia che ti contraddistingue. Vedrai che col tempo tutto si sistemerà.